

Introduction to R

Session 03: Data Visualisation

Álvaro Pérez

Instituto Tecnológico Autónomo de México

Spring 2025

Overview of Today's Session

GGplots

Customisation

Combination Plots

Overlapping Discrete Data

Overlapping Continuous Data

Interactive Plots

Types of variables

1. **Continuous**: properties you can measure, like height
2. **Discrete**: things you can count, like the number of pets you have
3. **Categorical**:
 - 3.1 **Nominal**: where the categories don't really have an order, like cats, dogs and ferrets.
 - 3.2 **Ordinal**: where there is a clear order, but the distance between the categories isn't something you could exactly equate, like points on a Likert rating scale.

Load the pets dataset from the dataskills package and explore it with `glimpse(pets)`

- ▶ R has some basic plotting functions, but they're difficult to use and aesthetically not very nice.
- ▶ They can be useful to have a quick look at data while you're working on a script, though.
- ▶ The function `plot()` usually defaults to a sensible type of plot, depending on whether the arguments `x` and `y` are categorical, continuous, or missing.

```
1 plot(x = pets$pet)
2
3 plot(x = pets$pet, y = pets$score)
4
5 plot(x = pets$age, y = pets$weight)
6
7 hist(pets$score, breaks = 20)
```

- ▶ The package `ggplot2` (loaded with `tidyverse`) is one of the most common packages for creating beautiful visualisations.
- ▶ `ggplot2` creates plots using a “grammar of graphics” where you add geoms in layers.

```
1 ggplot()
```

1. The first argument to `ggplot()` is the data table you want to plot.
2. The second argument is the mapping for which columns in your data table correspond to which properties of the plot, such as the x-axis, the y-axis, line colour or linetype, point shape, or object fill. These mappings are specified by the **`aes()`** function.

Example

```
1 mapping <- aes(x = pet,  
2                 y = score,  
3                 colour = country,  
4                 fill = country)  
5  
6 ggplot(data = pets, mapping = mapping)
```

- ▶ We usually omit the argument names and just put the `aes()` function directly as the second argument to `ggplot`.
- ▶ We also usually omit `x` and `y` as argument names to `aes()` (but you have to name the other properties).
- ▶ Next we can add “geoms,” or plot styles. You literally add them with the `+` symbol.
- ▶ You can also add other plot attributes, such as labels, or change the theme and base font size.

Example

```
1 ggplot(pets, aes(pet, score, colour = country, fill =  
2   country))  
3   geom_violin(alpha = 0.5) +  
4   labs(x = "Pet_type",  
5       y = "Score_on_an_Important_Test",  
6       colour = "Country_of_Origin",  
7       fill = "Country_of_Origin",  
8       title = "My_first_plot!") +  
9   theme_bw(base_size = 15)
```

Bar plot: good for categorical data where you want to represent the count.

```
1 ggplot(pets, aes(pet)) +  
2   geom_bar()
```

Density plot: good for one continuous variable, but only if you have a fairly large number of observations.

```
1 ggplot(pets, aes(score)) +  
2   geom_density()
```

- You can represent subsets of a variable by assigning the category variable to the argument group, fill, or color.

```
1 ggplot(pets, aes(score, fill = pet)) +  
2   geom_density(alpha = 0.5)
```


Frequency polygons: If you want the y-axis to represent count rather than density, try `geom_freqpoly()`.

```
1 ggplot(pets, aes(score, color = pet)) +  
2   geom_freqpoly(binwidth = 5)
```

Histogram: good for one continuous variable, and work well if you don't have many observations. Set the `binwidth` to control how wide each bar is.

```
1 ggplot(pets, aes(score)) +  
2   geom_histogram(binwidth = 5, fill = "white", color =  
   "black")
```

If you show grouped histograms, you also probably want to change the default position argument.

```
1 ggplot(pets, aes(score, fill=pet)) +  
2   geom_histogram(binwidth = 5, alpha = 0.5,  
3                 position = "dodge")
```

Box plot: great for representing the distribution of grouped continuous variables.

```
1 ggplot(pets, aes(pet, score, fill=pet)) +  
2   geom_boxplot(alpha = 0.5)
```

Violin plot: like sideways, mirrored density plots. They give even more information than a boxplot about distribution and are especially useful when you have non-normal distributions.

```
1 ggplot(pets, aes(pet, score, fill=pet)) +  
2   geom_violin(draw_quantiles = .5,  
3               trim = FALSE, alpha = 0.5,)
```

Scatter plot: scatter plots are a good way to represent the relationship between two continuous variables.

```
1 ggplot(pets, aes(age, score, color = pet)) +  
2   geom_point()
```

Line graph: you often want to represent the relationship as a single line.

```
1 ggplot(pets, aes(age, score, color = pet)) +  
2   geom_smooth(formula = y ~ x, method="lm")
```

You can set custom titles and axis labels in a few different ways.

```
1 ggplot(pets, aes(age, score, color = pet)) +  
2   geom_smooth(formula = y ~ x, method="lm") +  
3   labs(title = "Pet_score_with_Age",  
4         x = "Age_(in_Years)",  
5         y = "score_Score",  
6         color = "Pet_Type")
```

```
1 ggplot(pets, aes(age, score, color = pet)) +  
2   geom_smooth(formula = y ~ x, method="lm") +  
3   ggtitle("Pet_score_with_Age") +  
4   xlab("Age_(in_Years)") +  
5   ylab("score_Score") +  
6   scale_color_discrete(name = "Pet_Type")
```

You can set custom values for colour and fill using functions like `scale_colour_manual()` and `scale_fill_manual()`. [Here](#) you can find more R colours

```
1 ggplot(pets, aes(pet, score, colour = pet, fill = pet))  
  ) +  
2   geom_violin() +  
3   scale_color_manual(values = c("darkgreen", "  
    dodgerblue", "orange")) +  
4   scale_fill_manual(values = c("#CCFFCC", "#BBDDFF", "  
    #FFCC66"))
```

```
1 ggplot(pets, aes(age, score, color = pet)) +  
2   geom_smooth(formula = y ~ x, method="lm") +  
3   theme_minimal(base_size = 18)
```

You can save a ggplot using `ggsave()`. It saves the last ggplot you made, by default, but you can specify which plot you want to save if you assigned that plot to a variable.

```
1 box <- ggplot(pets, aes(pet, score, fill=pet)) +  
2   geom_boxplot(alpha = 0.5)  
3 ggsave("demog_box_plot.jpg", plot = box, width = 5,  
        height = 7)
```

Violinbox plot: a violin plot to show the shape of the distribution and a boxplot to show the median and interquartile ranges.

```
1 ggplot(pets, aes(pet, score, fill = pet)) +  
2   geom_violin(show.legend = FALSE) +  
3   geom_boxplot(width = 0.2, fill = "white",  
4               show.legend = FALSE)
```

Violin-point-range plot: You can use `stat_summary()` to superimpose a point-range plot showing the mean $\pm$ 1 SD.

```
1 ggplot(pets, aes(pet, score, fill=pet)) +  
2   geom_violin(trim = FALSE, alpha = 0.5) +  
3   stat_summary(  
4     fun = mean,  
5     fun.max = function(x) {mean(x) + sd(x)},  
6     fun.min = function(x) {mean(x) - sd(x)},  
7     geom="pointrange"  
8   )
```

Violin-jitter plot: If you don't have a lot of data points, it's good to represent them individually. You can use `geom_jitter` to do this.

```
1 # sample_n chooses 50 random observations from the
   dataset
2 ggplot(sample_n(pets, 50), aes(pet, score, fill=pet))
   +
3   geom_violin(
4     trim = FALSE,
5     draw_quantiles = c(0.25, 0.5, 0.75),
6     alpha = 0.5
7   ) +
8   geom_jitter(
9     width = 0.15, # points spread out over 15% of
   available width
10    height = 0, # do not move position on the y-axis
11    alpha = 0.5,
12    size = 3
13  )
```


Scatter-line graph: If your graph isn't too complicated, it's good to also show the individual data points behind the line.

```
1 ggplot(sample_n(pets, 50), aes(age, weight, colour =  
  pet)) +  
2   geom_point() +  
3   geom_smooth(formula = y ~ x, method="lm")
```

Grid of plots: You can use the **cowplot** package to easily make grids of different graphs. First, you have to assign each plot a name. Then you list all the plots as the first arguments of `plot_grid()` and provide a vector of labels.

```
1 gg <- ggplot(pets, aes(pet, score, colour = pet))
2 nolegend <- theme(legend.position = 0)
3
4 vp <- gg + geom_violin(alpha = 0.5) + nolegend +
5   ggtitle("Violin_Plot")
6 bp <- gg + geom_boxplot(alpha = 0.5) + nolegend +
7   ggtitle("Box_Plot")
8 cp <- gg + stat_summary(fun = mean, geom = "col", fill
9   = "white") + nolegend +
10  ggtitle("Column_Plot")
11 dp <- ggplot(pets, aes(score, colour = pet)) +
12  geom_density() + nolegend +
13  ggtitle("Density_Plot")
14 plot_grid(vp, bp, cp, dp, labels = LETTERS[1:4])
```

Overlapping Discrete Data

Reducing Opacity

You can deal with overlapping data points by reducing the opacity of the points. You need to use trial and error to adjust these so they look right.

```
1 ggplot(pets, aes(age, score, colour = pet)) +  
2   geom_point(alpha = 0.25) +  
3   geom_smooth(formula = y ~ x, method="lm")
```

Overlapping Discrete Data

Proportional Dot Plots

Or you can set the size of the dot proportional to the number of overlapping observations using `geom_count()`.

```
1 ggplot(pets, aes(age, score, colour = pet)) +  
2   geom_count()
```

2D Density plot: Use `geom_density2d()` to create a contour map.

```
1 ggplot(pets, aes(age, score)) +  
2   geom_density2d()
```

You can use `stat_density_2d(aes(fill = ..level..), geom = "polygon")` to create a heatmap-style density plot.

```
1 ggplot(pets, aes(age, score)) +  
2   stat_density_2d(aes(fill = ..level..), geom = "  
   polygon") +  
3   scale_fill_viridis_c()
```

2D Histogram: Use `geom_bin2d()` to create a rectangular heatmap of bin counts. Set the `binwidth` to the x and y dimensions to capture in each box.

```
1 ggplot(pets, aes(age, score)) +  
2   geom_bin2d(binwidth = c(1, 5))
```

Overlapping Continuous Data

Hexagonal Heatmap: Use `geom_hex()` to create a hexagonal heatmap of bin counts. Adjust the `binwidth`, `xlim()`, `ylim()` and/or the figure dimensions to make the hexagons more or less stretched.

```
1 ggplot(pets, aes(age, score)) +  
2   geom_hex(binwidth = c(1, 5))
```

Correlation Heatmap: Once you have a correlation matrix in the correct (long) format, it's easy to make a heatmap using `geom_tile()`.

```
1 heatmap <- pets %>%  
2   select_if(is.numeric) %>% # get just the numeric  
   columns  
3   cor() %>% # create the correlation matrix  
4   as_tibble(rownames = "V1") %>% # make it a tibble  
5   gather("V2", "r", 2:ncol()) # wide to long (V2)  
6 ggplot(heatmap, aes(V1, V2, fill=r)) +  
7   geom_tile() +  
8   scale_fill_viridis_c()
```

You can use the **plotly** package to make interactive graphs. Just assign your ggplot to a variable and use the function `ggplotly()`.

```
1 demog_plot <- ggplot(pets, aes(age, score, fill=pet))  
  +  
2   geom_point() +  
3   geom_smooth(formula = y~x, method = lm)  
4  
5 ggplotly(demog_plot)
```